



Introduction to JSON Web Tokens

Level up! Get the JWT Handbook for free and learn JWTs in depth!

What is JSON Web Token?

JSON Web Token (JWT) is an open standard ([RFC 7519](#)) that defines a compact and self-contained way for securely transmitting information between parties as a JSON object. This information can be verified and trusted because it is digitally signed. JWTs can be signed using a secret (with the **HMAC** algorithm) or a public/private key pair using **RSA** or **ECDSA**.

Although JWTs can be encrypted to also provide secrecy between parties, we will focus on *signed* tokens. Signed tokens can verify the *integrity* of the claims contained within it, while encrypted tokens *hide* those claims from other parties. When tokens are signed using public/private key pairs, the signature also certifies that only the party holding the private key is the one that signed it.

When should you use JSON Web Tokens?

We use cookies to provide the best website experience and to help understand marketing efforts. We may also share data with ad partners to reach potential customers across the web. To learn more, visit our **privacy policy**. Click **here for Your Privacy Choices**. You may also opt-out of this sharing by signaling your preference via GPC, applicable only to the browser signaling the opt-out.

[Cookie Settings](#)

Share feedback on new UI/UX ↗



JWT
Debugger

Additionally, as the signature is calculated using the header and the payload, you can also verify that the content hasn't been tampered with.

What is the JSON Web Token structure?

In its compact form, JSON Web Tokens consist of three parts separated by dots (.), which are:

- Header
- Payload
- Signature

Therefore, a JWT typically looks like the following:

```
xxxxx.yyyyy.zzzzz
```

Let's break down the different parts.

Header

The header *typically* consists of two parts: the type of the token, which is JWT, and the

We use cookies to provide the best website experience and to help understand marketing efforts. We may also share data with ad partners to reach potential customers across the web. To learn more, visit our **privacy policy**. Click **here for Your Privacy Choices**. You may also opt-out of this sharing by signaling your preference via GPC, applicable only to the browser signaling the opt-out.

[Share feedback on new UI/UX ↗](#)**JWT**
Debugger

The second part of the token is the payload, which contains the claims. Claims are statements about an entity (typically, the user) and additional data. There are three types of claims: *registered*, *public*, and *private* claims.

- **Registered claims**: These are a set of predefined claims which are not mandatory but recommended, to provide a set of useful, interoperable claims. Some of them are: `iss` (issuer), `exp` (expiration time), `sub` (subject), `aud` (audience), and [others](#).

Notice that the claim names are only three characters long, as JWT is meant to be compact.

- **Public claims**: These can be defined at will by those using JWTs. But to avoid collisions, they should be defined in the [IANA JSON Web Token Registry](#) or be defined as a URI that contains a collision-resistant namespace.
- **Private claims**: These are the custom claims created to share information between parties that agree on using them and are neither *registered* or *public* claims.

An example payload could be:

We use cookies to provide the best website experience and to help understand marketing efforts. We may also share data with ad partners to reach potential customers across the web. To learn more, visit our **privacy policy**. Click **here for Your Privacy Choices**. You may also opt-out of this sharing by signaling your preference via GPC, applicable only to the browser signaling the opt-out.

[Share feedback on new UI/UX ↗](#)**JWT**
Debugger

Signature

To create the signature part you have to take the encoded header, the encoded payload, a secret, the algorithm specified in the header, and sign that.

For example, if you want to use the HMAC SHA256 algorithm, the signature will be created in the following way:

```
HMACSHA256(  
  base64UrlEncode(header) + "." +  
  base64UrlEncode(payload),  
  secret)
```

The signature is used to verify the message wasn't changed along the way, and, in the case of tokens signed with a private key, it can also verify that the sender of the JWT is who it says it is.

Putting all together

The output is three Base64-URL strings separated by dots that can be easily passed in HTML and HTTP environments, while being more compact when compared to XML-based standards

We use cookies to provide the best website experience and to help understand marketing efforts. We may also share data with ad partners to reach potential customers across the web. To learn more, visit our **privacy policy**. Click **here for Your Privacy Choices**. You may also opt-out of this sharing by signaling your preference via GPC, applicable only to the browser signaling the opt-out.

[Share feedback on new UI/UX ↗](#)**JWT**
Debugger

gRG9II1w1aXN1b2NpYWw1OnRydWV9.
4pcPyMD09o1PSyXnrXCjTwXyr4BsezDI1AVTmud2fU4

If you want to play with JWT and put these concepts into practice, you can use jwt.io [Debugger](#) to decode, verify, and generate JWTs.

We use cookies to provide the best website experience and to help understand marketing efforts. We may also share data with ad partners to reach potential customers across the web. To learn more, visit our **privacy policy**. Click **here for Your Privacy Choices**. You may also opt-out of this sharing by signaling your preference via GPC, applicable only to the browser signaling the opt-out.



JWT
Debugger



HS256

Decoded

HEADER:

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

PAYLOAD:

```
{
  "sub": "1234567890",
  "name": "John Doe",
  "iat": 1516239022
}
```

VERIFY SIGNATURE

```
HMACSHA256(
    base64UrlEncode(header) + "." +
    base64UrlEncode(payload),
    your-256-bit-secret
) ☐ secret base64 encoded
```

<https://jwt.io/introduction>

[Share feedback on new UI/UX ↗](#)

You also [should not store sensitive session data in browser storage due to lack of security](#).

Whenever the user wants to access a protected route or resource, the user agent should send the JWT, typically in the **Authorization** header using the **Bearer** schema. The content of the header should look like the following:

```
Authorization: Bearer <token>
```

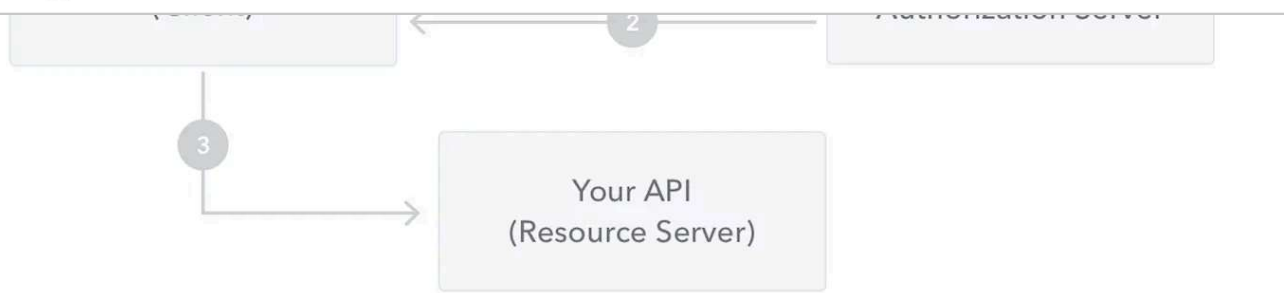
This can be, in certain cases, a stateless authorization mechanism. The server's protected routes will check for a valid JWT in the Authorization header, and if it's present, the user will be allowed to access protected resources. If the JWT contains the necessary data, the need to query the database for certain operations may be reduced, though this may not always be the case.

Note that if you send JWT tokens through HTTP headers, you should try to prevent them from getting too big. Some servers don't accept more than 8 KB in headers. If you are trying to embed too much information in a JWT token, like by including all the user's permissions, you may need an alternative solution, like [Auth0 Fine-Grained Authorization](#).

If the token is sent in the Authorization header, Cross-Origin Resource Sharing (CORS) won't be an issue as it doesn't use cookies.

The following diagram shows how a JWT is obtained and used to access APIs or resources:

We use cookies to provide the best website experience and to help understand marketing efforts. We may also share data with ad partners to reach potential customers across the web. To learn more, visit our **privacy policy**. Click **here for Your Privacy Choices**. You may also opt-out of this sharing by signaling your preference via GPC, applicable only to the browser signaling the opt-out.

[Share feedback on new UI/UX ↗](#)JWT
Debugger

1. The application or client requests authorization to the authorization server. This is performed through one of the different authorization flows. For example, a typical [OpenID Connect](#) compliant web application will go through the /oauth/authorize endpoint using the [authorization code flow](#).
2. When the authorization is granted, the authorization server returns an access token to the application.
3. The application uses the access token to access a protected resource (like an API).

Do note that with signed tokens, all the information contained within the token is exposed to users or other parties, even though they are unable to change it. This means you should not put secret information within the token.

Why should we use JSON Web Tokens?

We use cookies to provide the best website experience and to help understand marketing efforts. We may also share data with ad partners to reach potential customers across the web. To learn more, visit our [privacy policy](#). Click **here for Your Privacy Choices**. You may also opt-out of this sharing by signaling your preference via GPC, applicable only to the browser signaling the opt-out.

[Share feedback on new UI/UX ↗](#)**JWT**
Debugger

JSON parsers are common in most programming languages because they map directly to objects. Conversely, XML doesn't have a natural document-to-object mapping. This makes it easier to work with JWT than SAML assertions.

Regarding usage, JWT is used at the Internet scale. This highlights the ease of client-side processing of the JSON Web token on multiple platforms, especially mobile.

We use cookies to provide the best website experience and to help understand marketing efforts. We may also share data with ad partners to reach potential customers across the web. To learn more, visit our **privacy policy**. Click **here for Your Privacy Choices**. You may also opt-out of this sharing by signaling your preference via GPC, applicable only to the browser signaling the opt-out.



JWT
Debugger



JWT

Debugger

ETC00000 PASTE A TOKEN HERE

DECRYPT EDIT THE PAYLOAD AND SECRET (ONLY HS256 SUPPORTED)

HEADER: ALGORITHM & TOKEN TYPE

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

PAYLOAD: DATA

```
{
  "sub": "1234567890",
  "name": "John Doe",
  "admin": true
}
```

VERIFY SIGNATURE

```
HMACSHA256(
    base64UrlEncode(header) + "." +
    base64UrlEncode(payload),
    secret
) // secret base64 encoded
```

VS



<https://jwt.io/introduction>

[Share feedback on new UI/UX ↗](#)**JWT**
Debugger

Comparison of the length of an encoded JWT and an encoded SAML

If you want to read more about JSON Web Tokens and even start using them to perform authentication in your own applications, browse to the [JSON Web Token landing page](#) at Auth0.

Difference Between Validating and Verifying a JWT

JSON Web Token (JWT) validation and verification are crucial for security, but they address slightly different aspects of JWT security: validation ensures the token is well-formed and contains enforceable claims; verification ensures the token is genuine and unmodified.

Let's explore in more detail how validation and verification differ:

JWT validation generally refers to checking the structure, format, and content of the JWT:

- **Structure:** Ensuring the token has the standard three parts (header, payload, signature) separated by dots.
- **Format:** Verifying that each part is correctly encoded (Base64URL) and that the payload contains expected claims.

Content: Checking if the claims within the payload are correct, such as expiration time

We use cookies to provide the best website experience and to help understand marketing efforts. We may also share data with ad partners to reach potential customers across the web. To learn more, visit our **privacy policy**. Click **here for Your Privacy Choices**. You may also opt-out of this sharing by signaling your preference via GPC, applicable only to the browser signaling the opt-out.

[Share feedback on new UI/UX ↗](#)**JWT**
Debugger

- **Issuer Verification:** Checking if the iss claim matches an expected issuer.
- **Audience Check:** Ensuring the aud claim matches the expected audience.

In practical terms:

You validate a JWT to make sure the token makes sense, adheres to the expected standards, contains the right data.

You verify a JWT to make sure the token hasn't been altered maliciously and comes from a trusted source.

In many systems, these steps are often combined into what might be colloquially called "JWT verification" which encompasses both validation and verification for comprehensive security checks. Nonetheless, their distinction remains.

Difference Between Decoding and Encoding a JWT

Encoding a JWT involves transforming the header and payload into a compact, URL-safe format. The header, which states the signing algorithm and token type, and the payload, which includes claims like subject, expiration, and issue time, are both converted to JSON then Base64URL encoded. These encoded parts are then concatenated with a dot, after which a signature is generated using the algorithm specified in the header with a secret or private key. This signature is also Base64URL encoded, resulting in the final JWT string that represents the

We use cookies to provide the best website experience and to help understand marketing efforts. We may also share data with ad partners to reach potential customers across the web. To learn more, visit our **privacy policy**. Click **here for Your Privacy Choices**. You may also opt-out of this sharing by signaling your preference via GPC, applicable only to the browser signaling the opt-out.

[Share feedback on new UI/UX ↗](#)**JWT**
Debugger

Get Started with JSON Web Tokens

Securely implement authentication with JWTs using Auth0 on any stack and any device in less than 10 minutes.

[Create a free account →](#)

We use cookies to provide the best website experience and to help understand marketing efforts. We may also share data with ad partners to reach potential customers across the web. To learn more, visit our **privacy policy**. Click **here for Your Privacy Choices**. You may also opt-out of this sharing by signaling your preference via GPC, applicable only to the browser signaling the opt-out.

Share feedback on new UI/UX ↗



JWT
Debugger

We use cookies to provide the best website experience and to help understand marketing efforts. We may also share data with ad partners to reach potential customers across the web. To learn more, visit our **privacy policy**. Click **here for Your Privacy Choices**. You may also opt-out of this sharing by signaling your preference via GPC, applicable only to the browser signaling the opt-out.

Share feedback on new UI/UX ↗



JWT
Debugger

LEGAL

[Privacy Policy](#)

[Security](#)

[Your Privacy Choices](#) 

SOCIAL



Copyright © 2025 Okta. All rights reserved.

We use cookies to provide the best website experience and to help understand marketing efforts. We may also share data with ad partners to reach potential customers across the web. To learn more, visit our [privacy policy](#). Click [here for Your Privacy Choices](#). You may also opt-out of this sharing by signaling your preference via GPC, applicable only to the browser signaling the opt-out.